

CSC3066 Deep Learning

Recurrent Neural Networks

Lu Bai (l.bai@qub.ac.uk)



Coming today

- ☐ Introduction to Recurrent Neural Networks (RNN)
- ☐ Architecture of RNN
- ☐ Training RNN
- ☐ Issues with “Vanilla” RNN
- ☐ Long Short-Term Memory RNN

Introduction to RNN



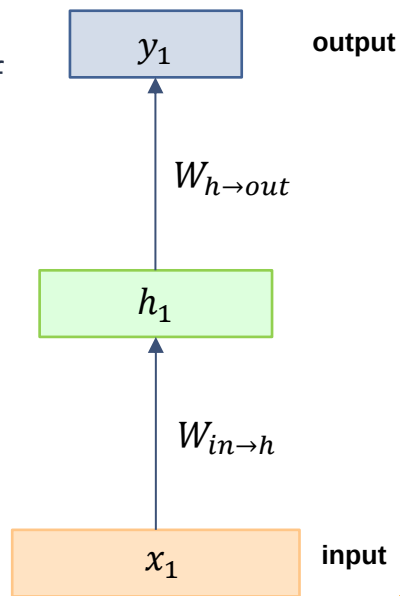
Introduction to RNN

- ☐ ANNs that we have seen so far (feed forward ANNs) process each input independently.
- ☐ ANNs can't use their reasoning about previous events. They start thinking "from scratch" every time you provide them with a new input.
- ☐ In order to process a sequence or a temporal series of data points, we have to show the entire sequence to the network at once.
- ☐ This is what we did with the sentiment analysis problem: an entire movie review was transformed into a large single vector or into a matrix.
- ☐ This is a big shortcoming in certain application.



Introduction to RNN

- Feed-forward Network
 - Input of fixed size (matrix, vector)
 - Single input is passed through a set of hidden layers and produces a single output (e.g. classification score)





Introduction to RNN

- ☐ In some contexts in ML we may want to have more flexibility in terms of the data type that the model can process.
- ☐ RNNs give us possibilities to apply different type of input-output data.
- ☐ RNNs are capable of processing sequences of events.
- ☐ RNNs are networks with loops in them, allowing information to persist.

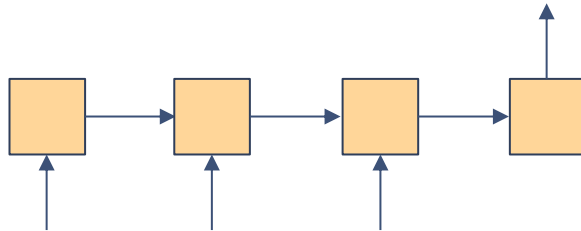


Introduction to RNN

- ❑ RNN Input-Output Scenarios
 - ❑ **RNNs** are designed to work with sequence prediction problems.
 - ❑ Sequence prediction problems come in many forms. They can be described by the types of inputs and outputs supported by RNN.

Introduction to RNN

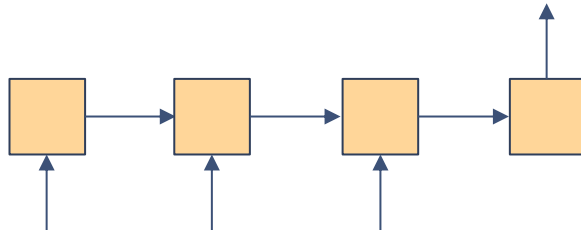
□ RNN Input-Output Scenarios



- **Many-to-One:** An input sequence mapped to class or quantity prediction, e.g. ?

Introduction to RNN

❑ RNN Input-Output Scenarios

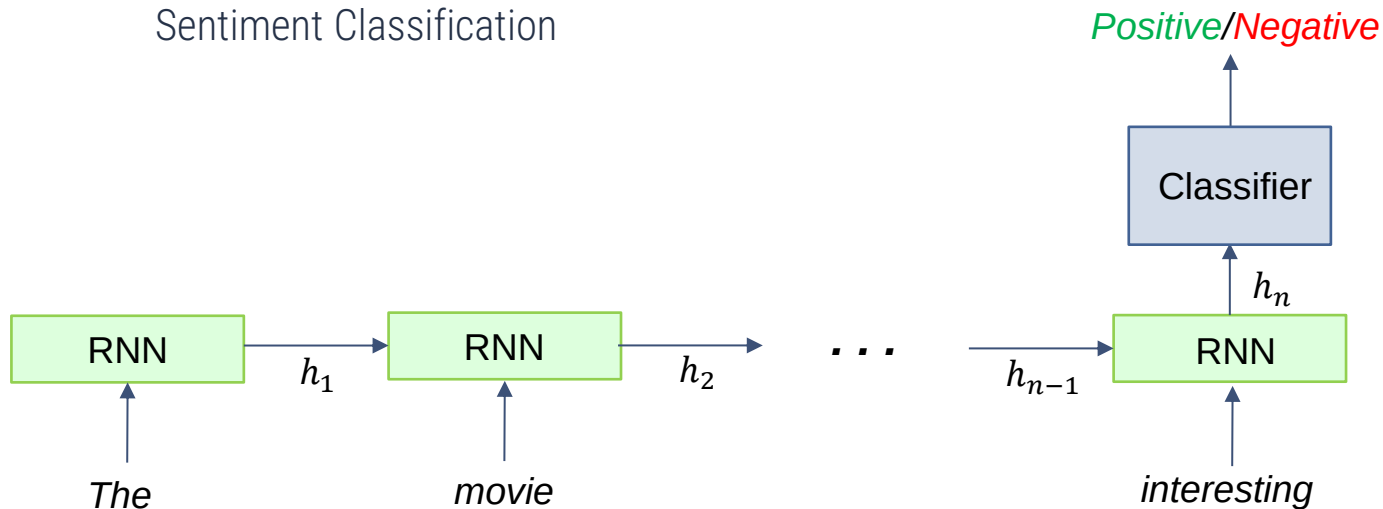


- ❑ **Many-to-One:** An input sequence mapped to class or quantity prediction, e.g. sentiment analysis, time series forecasting, stock price prediction

Introduction to RNN

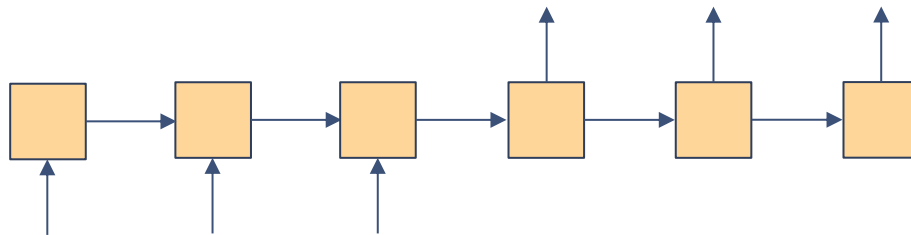
□ Many-to-One:

Sentiment Classification



Introduction to RNN

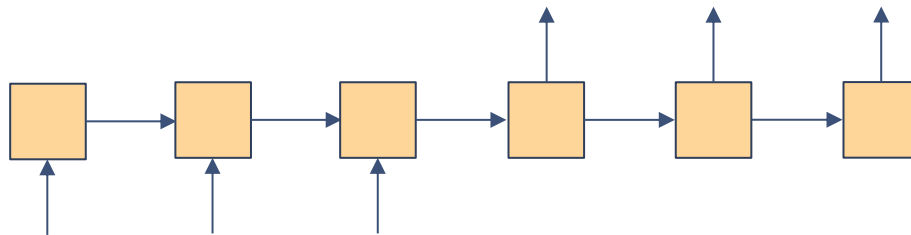
□ RNN Input-Output Scenarios



- **Many-to-Many:** An input sequence is mapped into output sequence, e.g. ?

Introduction to RNN

❑ RNN Input-Output Scenarios

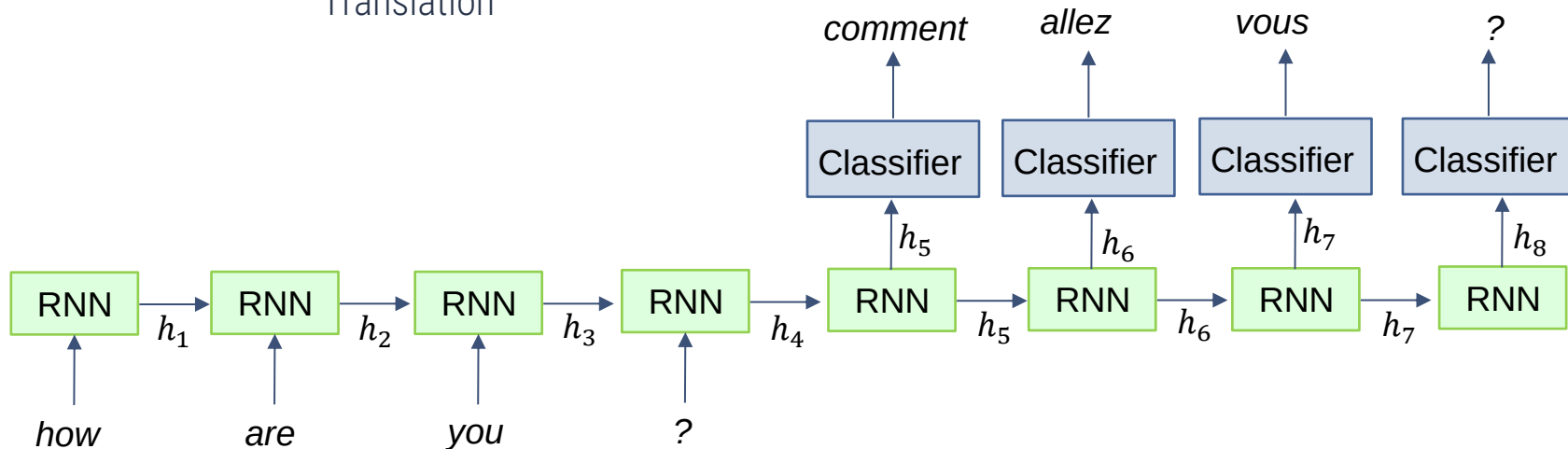


- ❑ **Many-to-Many:** An input sequence is mapped into output sequence, e.g. language translation, speech recognition.

Introduction to RNN

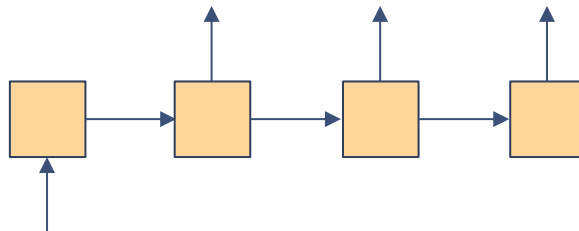
□ Many-to-Many:

Translation



Introduction to RNN

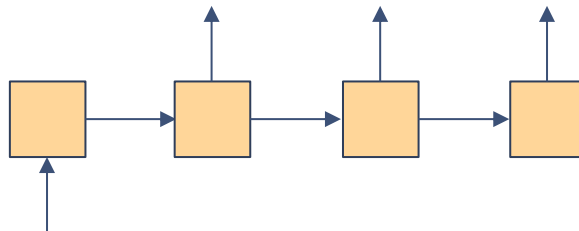
❑ RNN Input-Output Scenarios



- ❑ **One-to-Many:** A single input observation is mapped to a sequence as an output, e.g. ?

Introduction to RNN

❑ RNN Input-Output Scenarios

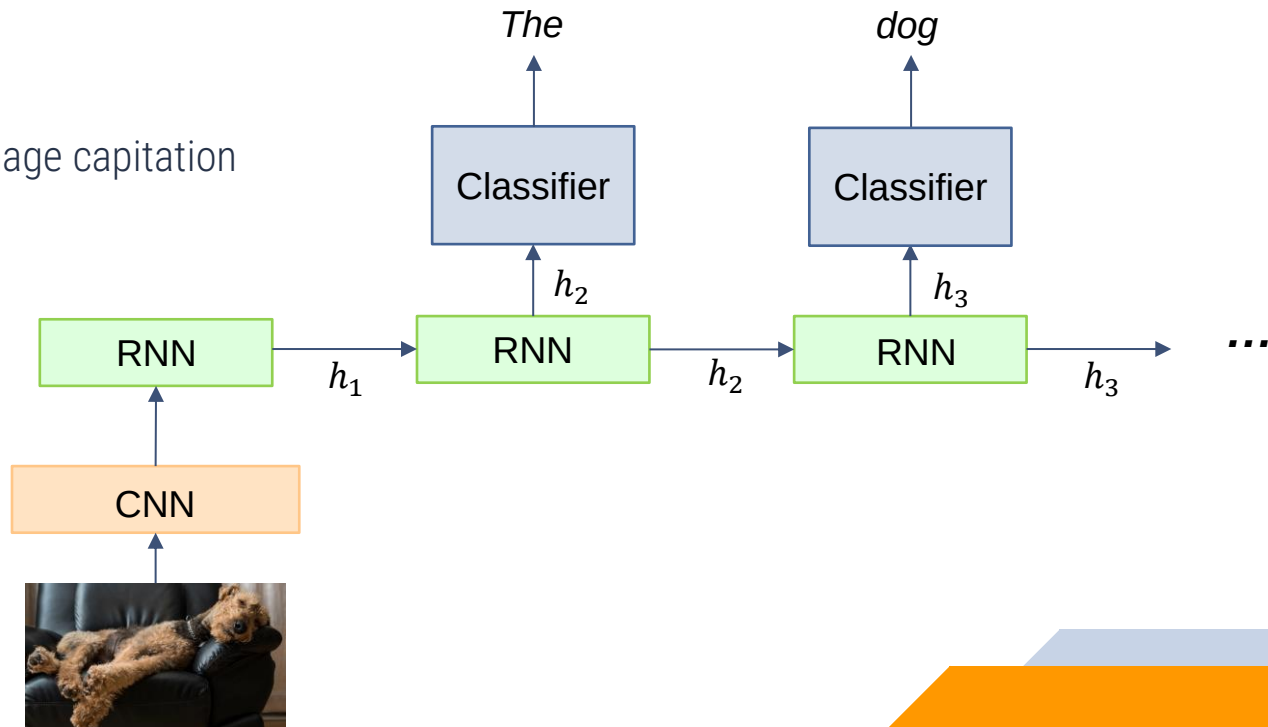


- ❑ **One-to-Many:** A single input observation is mapped to a sequence as an output, e.g. generating image description.

Introduction to RNN

One-to-Many:

Predicting image caption





Introduction to RNN

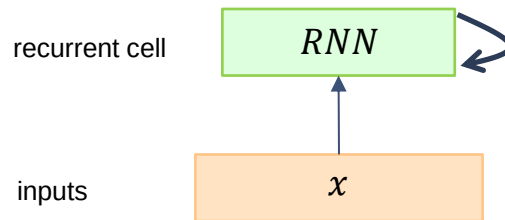
- ❑ In the last few years, there have been incredible success applying RNNs to a variety of problems:
 - Language translation
 - Time series forecasting problems
 - Speech recognition
 - Text classification
 - ...
- ❑ Great post about RNN and its capabilities (<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>)

Architecture of RNNs



RNN Architecture

- Architecture of RNNs
 - Input layer
 - Recurrent cell (layer) with internal hidden state which is updated every time RNN reads a new input
 - Internal hidden state is fed back to the model next time the model reads a new input





RNN Architecture

□ Architecture of RNN

- A sequence of vectors $X = \{x_1, x_2, \dots\}$ is processed by applying a recurrent formula at every time step t :

$$h_t = f_h(h_{t-1}W + x_tV + b_h)$$

h_t : new state

f_h : activation function

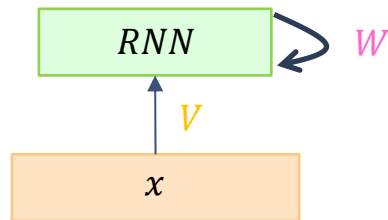
h_{t-1} : old state

W : weight matrix multiplied by previous hidden state h_{t-1}

x_t : input vector at time step t

V : weight matrix multiplied by input x_t

b_h : bias added when calculating h_t



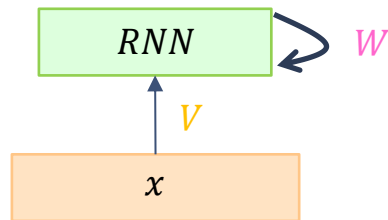


RNN Architecture

- Architecture of RNN
 - A sequence of vectors $X = \{x_1, x_2, \dots\}$ is processed by applying a recurrent formula at every time step t :

$$h_t = f_h(h_{t-1}W + x_tV + b_h)$$

- At every time step of the computation the same function f_h and the same weights W and V are used.





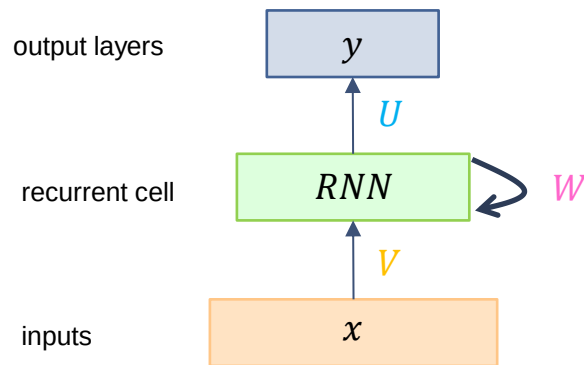
RNN Architecture

□ Architecture of RNN

- If we want to produce an output at every time step, we need to attach some fully connected layers to read the output h_t at every time step and make a decision based on it.

$$h_t = f_h(h_{t-1}W + x_tV + b_h)$$

$$y = f_y(h_tU + b_y)$$

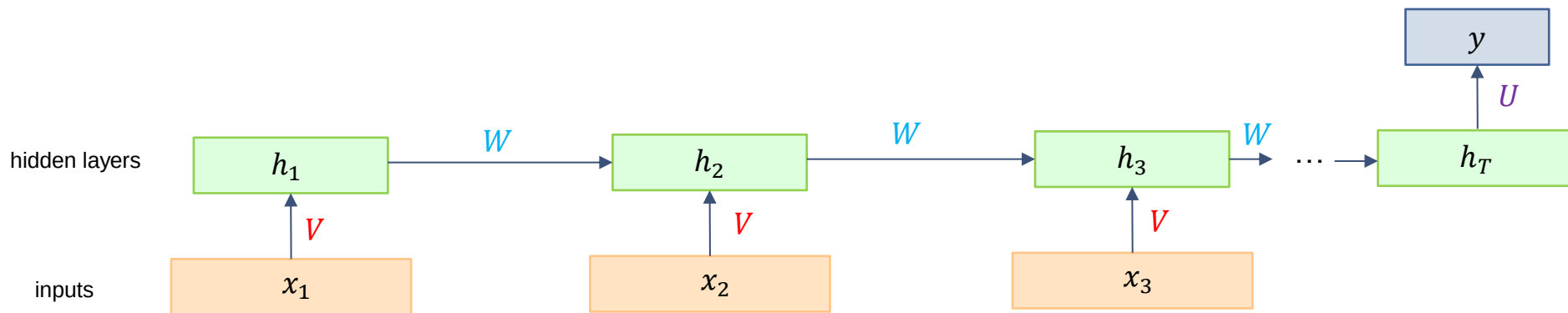




RNN Architecture

□ Architecture of RNN: Many to One

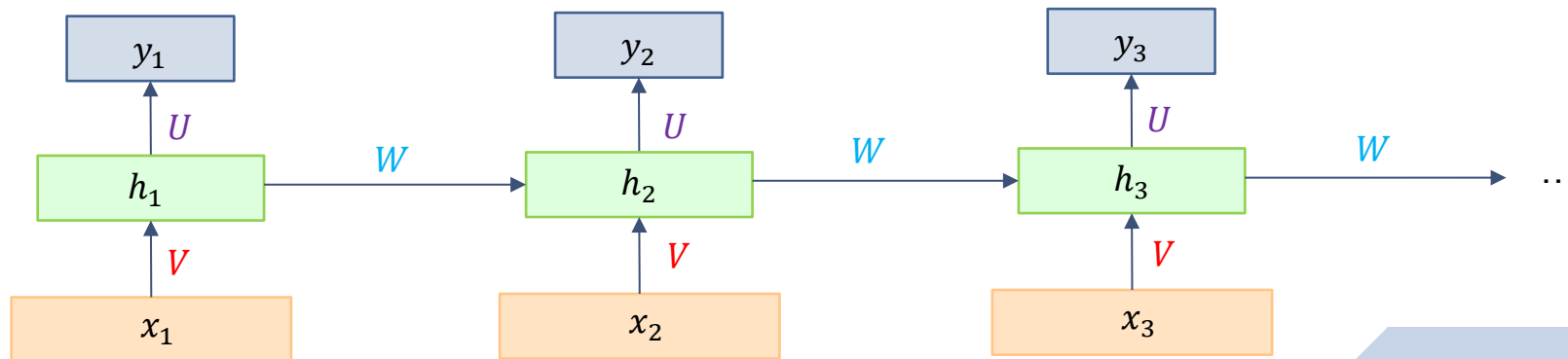
- At each time step t , the current input x_t and the hidden state from the previous time step h_{t-1} are used to produce the next hidden state h_t .





RNN Architecture

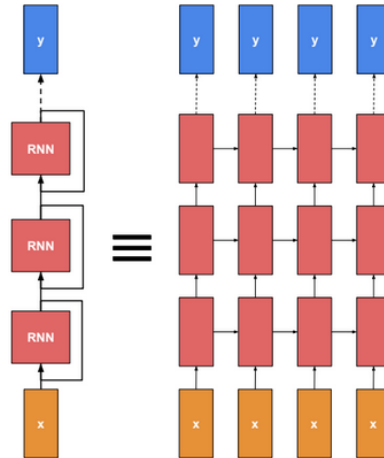
- Architecture of RNN: Many to Many
 - Output h_t at every time step is fed into another network, which can produce output y_t , which could be a class label for example.



RNN Architecture

□ Multilayer RNN:

- Deep RNNs can be created by stacking multiple RNN layers, with the output sequence of one layer forming the input sequence of the next (in addition to recurrent connections within the same layer)



Training RNN

RNN Architecture - Loss

- ❑ RNN: Calculating Loss (Many to Many)
 - ❑ Loss is calculated at every time step
 - ❑ The final loss is the sum of the individual losses

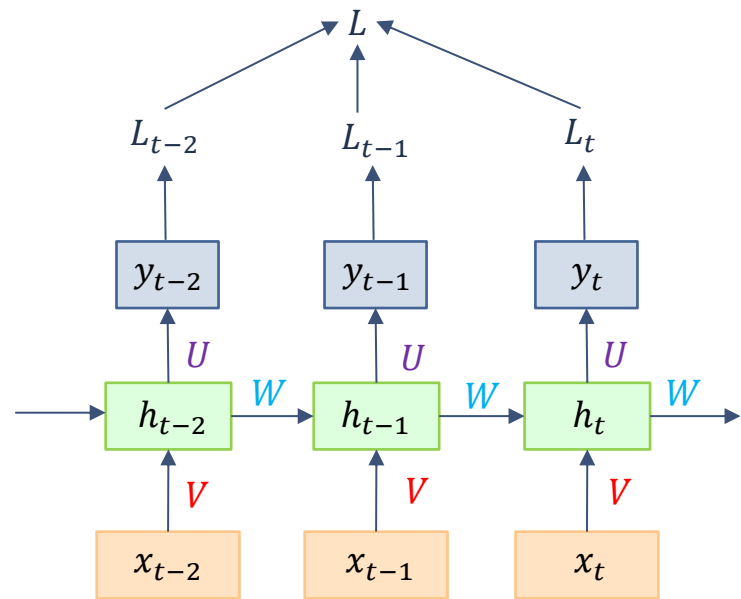
$\hat{y}_t$ –true label

y_t –output (prediction)

$L_t(y_t, \hat{y}_t)$ –some loss function

Final Loss:

$$L = \sum_i L_i(y_i, \hat{y}_i)$$

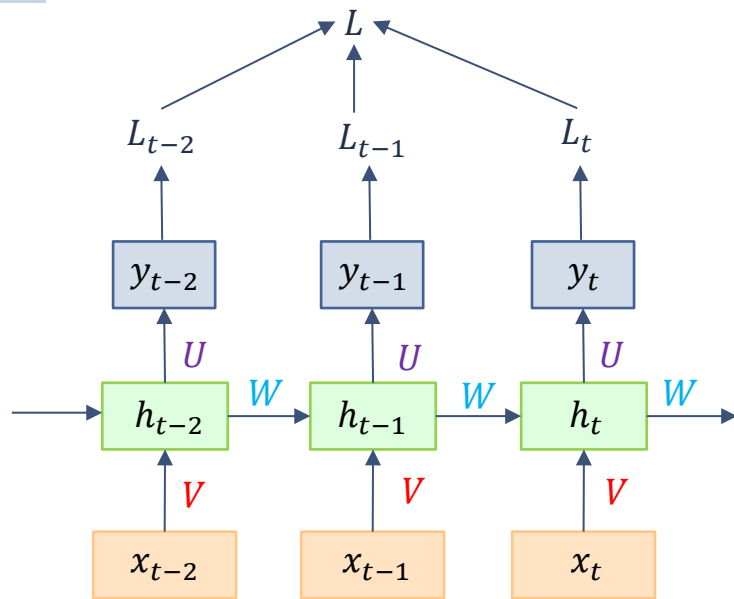


RNN Training

□ Backpropagation

- We need to calculate gradient with respect to all parameters:

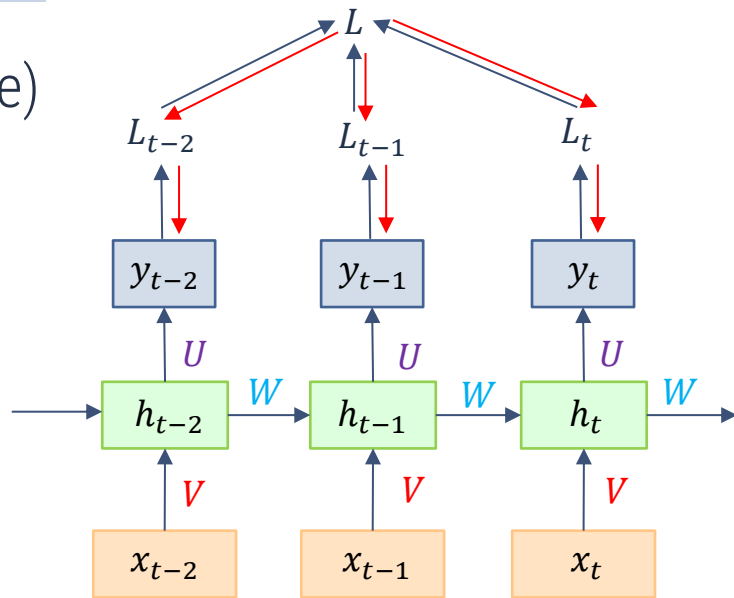
$$\frac{\partial L}{\partial U}, \frac{\partial L}{\partial V}, \frac{\partial L}{\partial W}, \frac{\partial L}{\partial b_y}, \frac{\partial L}{\partial b_h}$$



RNN Training

- Backpropagation (Many to Many case)

$$\frac{\partial L}{\partial U} = \sum_{i=1}^T \frac{\partial L_i}{\partial U}$$



RNN Training

- Backpropagation (Many to Many case)

$$\frac{\partial L}{\partial U} = \sum_{i=1}^T \frac{\partial L_i}{\partial U}$$

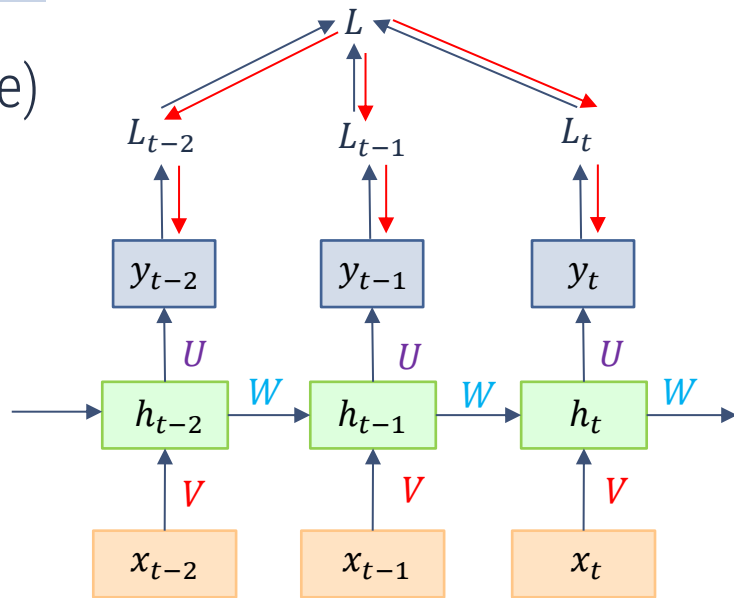
- We can use chain rule here:

$$\frac{\partial L_t}{\partial U} = \frac{\partial L_t}{\partial y_t} \frac{\partial y_t}{\partial U}$$

$$y_t = f_y(\textcolor{red}{U}h_t + b_y)$$



This is the only dependency



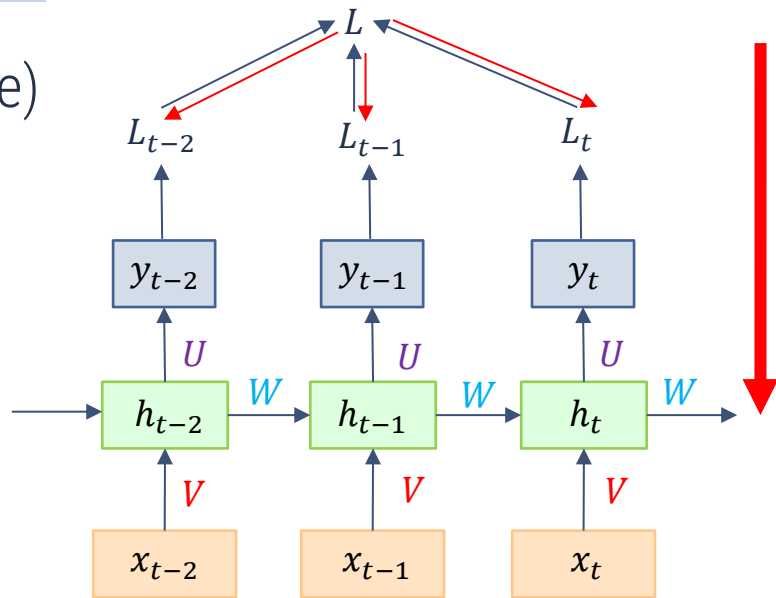
RNN Training

- Backpropagation (Many to Many case)

$$\frac{\partial L}{\partial W} = \sum_{i=1}^T \frac{\partial L_i}{\partial W}$$

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial y_t} \times \frac{\partial y_t}{\partial h_t} \times \frac{\partial h_t}{\partial W}$$

$$h_t = f_h(W h_{t-1} + V x_t + b_h)$$



RNN Training

Backpropagation (Many to Many case)

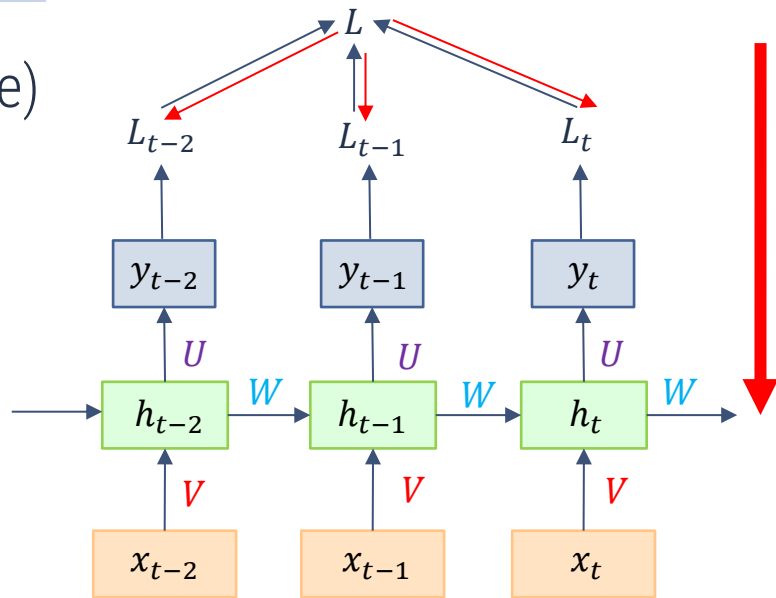
$$\frac{\partial L}{\partial W} = \sum_{i=1}^T \frac{\partial L_i}{\partial W}$$

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial y_t} \times \frac{\partial y_t}{\partial h_t} \times \frac{\partial h_t}{\partial W}$$

$$h_t = f_h(W h_{t-1} + V x_t + b_h)$$



Is it the only dependency?



RNN Training

□ Backpropagation (Many to Many case)

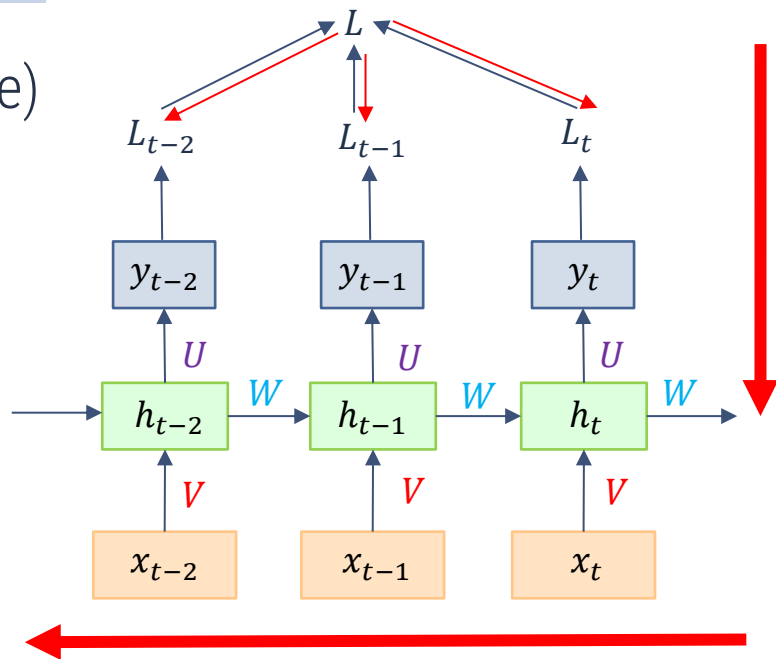
$$\frac{\partial L}{\partial W} = \sum_{i=1}^T \frac{\partial L_i}{\partial W}$$

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial y_t} \times \frac{\partial y_t}{\partial h_t} \times \frac{\partial h_t}{\partial W}$$

$$h_t = f_h(W h_{t-1} + V x_t + b_h)$$



This is NOT the only dependency This depends on W too!



RNN Training

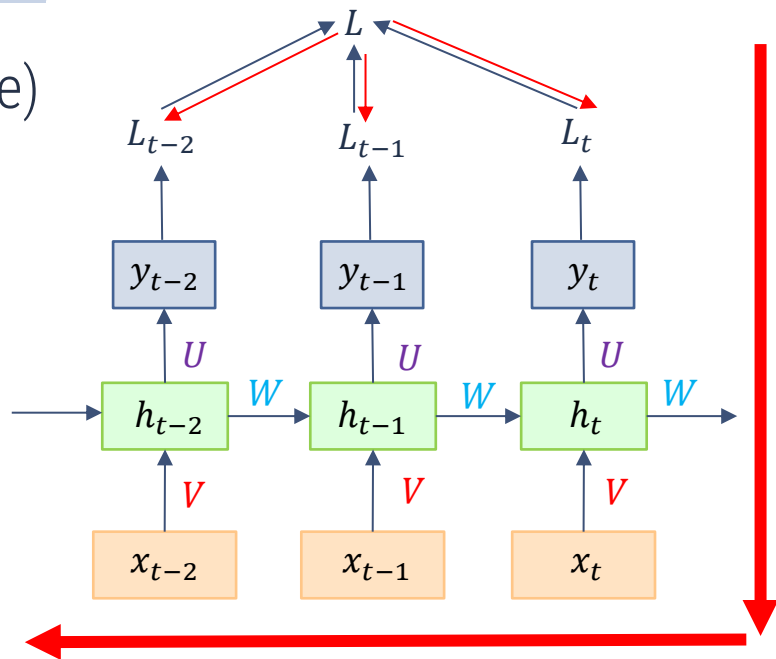
Backpropagation (Many to Many case)

$$\frac{\partial L}{\partial V} = \sum_{i=1}^T \frac{\partial L_i}{\partial V}$$

$$\frac{\partial L_t}{\partial V} = \frac{\partial L_t}{\partial y_t} \times \frac{\partial y_t}{\partial h_t} \times \frac{\partial h_t}{\partial V}$$

$$h_t = f_h(W h_{t-1} + V x_t + b_h)$$

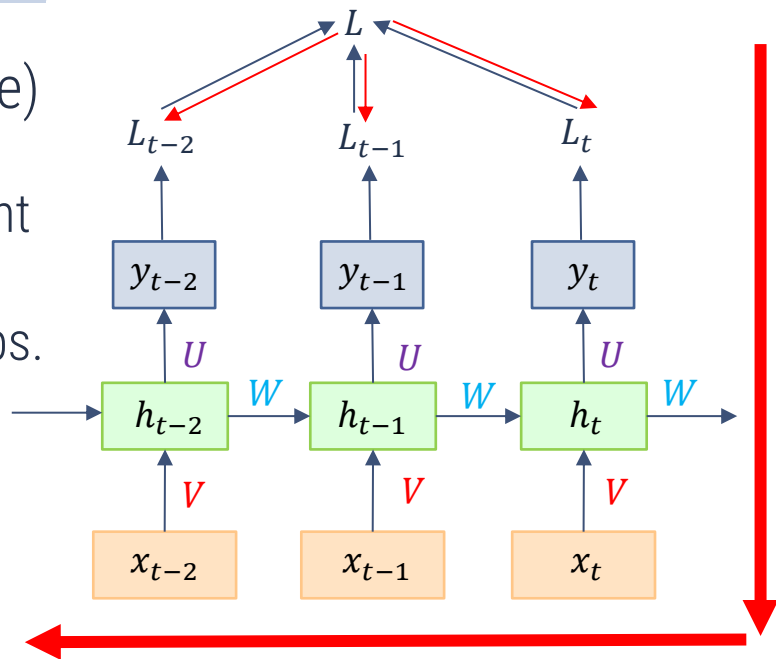
Depend on V too!



RNN Training

□ Backpropagation (Many to Many case)

- In order to fully calculate the gradient of W and V , we will need to backpropagate through all time steps.
- This is known as **backpropagation through time**.



RNN Training

- Backpropagation (Many to Many case)
 - The gradient of the Loss at the t time step is given by:

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial y_t} \times \frac{\partial y_t}{\partial h_t} \sum_{k=0}^t \left(\prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W}$$

RNN Training

- Backpropagation (Many to Many case)
 - The gradient of the Loss at the t time step is given by:

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial y_t} \times \frac{\partial y_t}{\partial h_t} \sum_{k=0}^t \left(\prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W}$$

$$h_i = f_h(h_{i-1}W + x_iV + b_h)$$

RNN Training

- Backpropagation (Many to Many case)
 - The gradient of the Loss at the t time step is given by:

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial y_t} \times \frac{\partial y_t}{\partial h_t} \sum_{k=0}^t \left(\prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W}$$

$$\frac{\partial h_i}{\partial h_{i-1}} = f_h'(h_{i-1}W + x_iV + b_h) \times \frac{\partial}{\partial h_{i-1}} [h_{i-1}W + x_iV + b_h] =$$

RNN Training

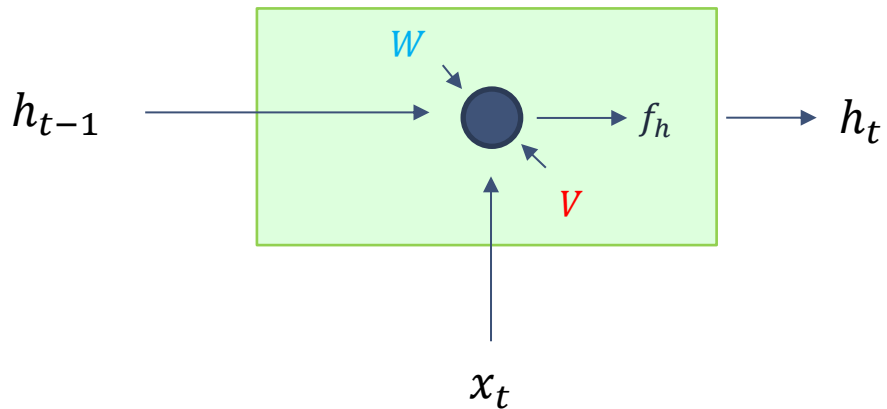
- Backpropagation (Many to Many case)
 - The gradient of the Loss at the t time step is given by:

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial y_t} \times \frac{\partial y_t}{\partial h_t} \sum_{k=0}^t \left(\prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W}$$

$$\frac{\partial h_i}{\partial h_{i-1}} = f_h'(h_{i-1}W + x_iV + b_h) \times W$$

RNN Training

- Backpropagation through time

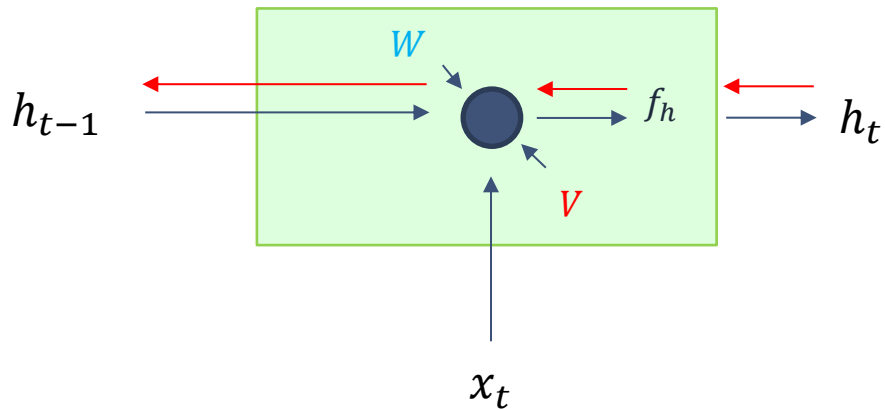


$$h_t = f_h(W h_{t-1} + V x_t + b_h)$$

RNN Training

□ Backpropagation through time

- Every time we backpropagate through one of the RNN cells we multiply the gradient by W and the derivative of f_h



$$h_t = f_h(W h_{t-1} + V x_t + b_h)$$

$$\frac{h_t}{h_{t-1}} = f_h'(h_{t-1}W + x_tV + b_h) \times W$$

RNN Training

- Backpropagation through time
 - Every time we backpropagate through one of the RNN cells we multiply the gradient by W
 - For many RNN cells the gradient will become very small (we already know this problem)

Issues with RNN

RNN: Vanishing Gradients

- Expression of the gradient:

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial y_t} \times \frac{\partial y_t}{\partial h_t} \sum_{k=0}^t \left(\prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W}$$

The more steps between the time moments k and t , the more elements are in this product.

RNN: Vanishing Gradients

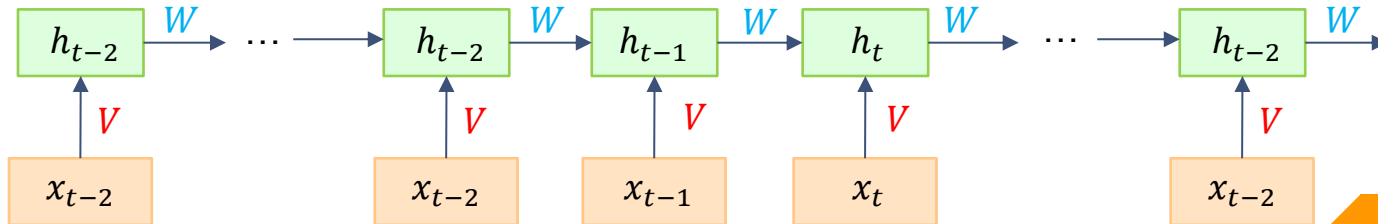
- ❑ The training signal is back-propagated through the network; however, its utility tends to diminish the further back we go.
- ❑ Gradient contributions from “far away” steps become zero, and the state at those steps doesn’t contribute to the learning process.
- ❑ This means the network is not learning long-range dependencies.
- ❑ The model is highly sensitive to the immediately preceding input, rather than input from much further back.

RNN: Vanishing Gradients

- ❑ Vanishing gradients also happen in deep feed-forward NNs.
- ❑ RNNs tend to be very deep (as deep as the sentence or document length in our case), which makes the problem a lot more common.
- ❑ This is a problem for modelling language, where information from a long way back may still be important.

RNN: Short Memory

- ❑ Recurrent Neural Networks suffer from short-term memory.
- ❑ If a sequence is long enough, they have a hard time carrying information from earlier time steps to later ones.
- ❑ Words from time steps far away are not taken into consideration when training to make a predict (e.g. the next word).



RNN: Short Memory

- ❑ Assume we want to train a RNN that can predict a missing word in a sequence.
- ❑ Sometimes, we only need to look at recent information to perform the present task.
 - ❑ The grass colour is “green”. -> the gap between the prediction “green” and the necessary context information “grass” is small.
- ❑ But there are also cases where we need more context.
 - ❑ Jane walked into the room. John walked in too. It was late in the day. John said hi to ____

LSTM

Long Short Term Memory

LSTM Recurrent Neural Network

- ❑ LSTM: Long-Short Term Memory
 - ❑ LSTM are the types of RNN that allow to establish long term dependencies.
 - ❑ LSTMs are explicitly designed to avoid the long-term dependency problem.
 - ❑ They work better in comparison to the simple RNN since they overcome vanishing gradient problem.

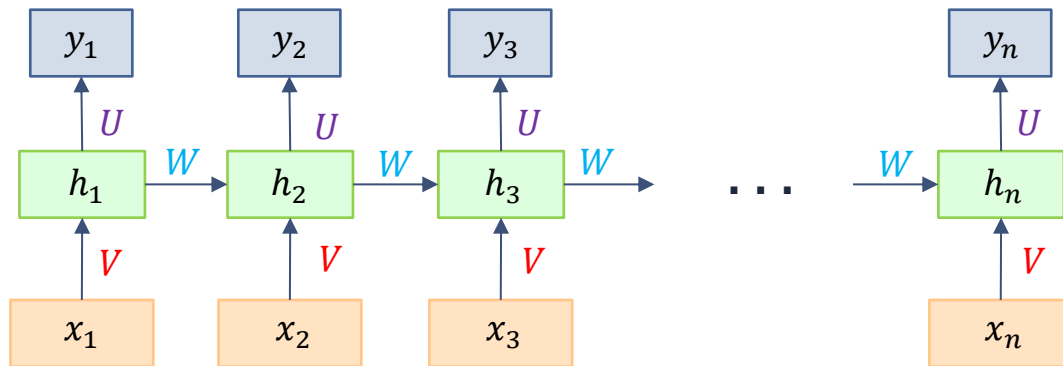
LSTM Recurrent Neural Network

- ❑ LSTM: Long-Short Term Memory

- ❑ All recurrent neural networks have the form of a chain of repeating modules. In standard RNNs, this repeating module will have a very simple structure, such as a single *tanh* layer.

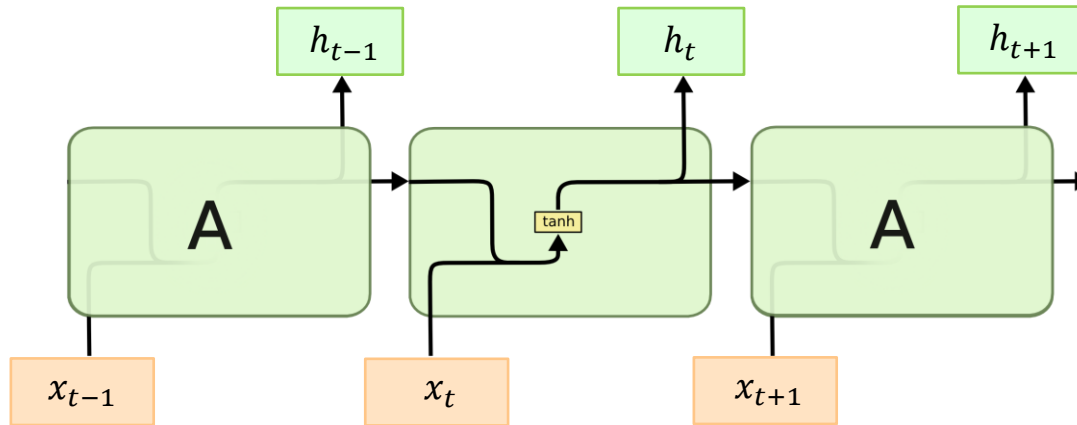
LSTM Recurrent Neural Network

□ Simple RNN architecture



LSTM Recurrent Neural Network

□ Simple RNN architecture

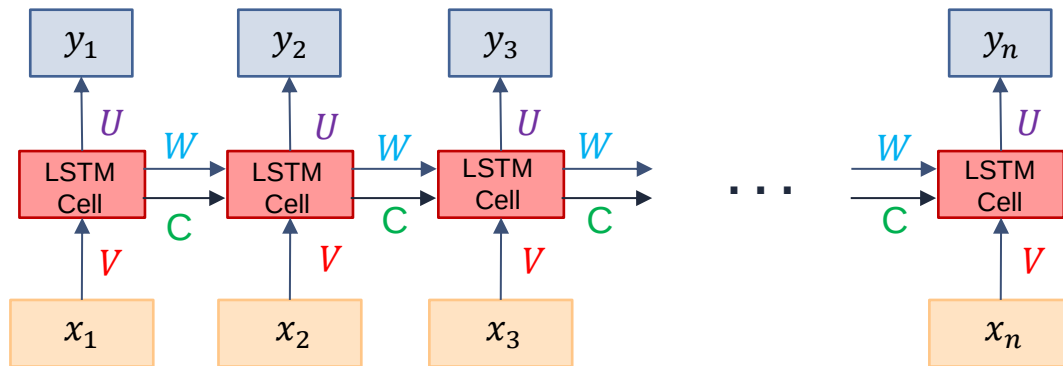


LSTM Recurrent Neural Network

- ❑ LSTM: Long-Short Term Memory
 - ❑ LSTMs also have this chain like structure, but the repeating module has a different structure. Instead of having a single neural network layer, there are four, interacting in a very special way.

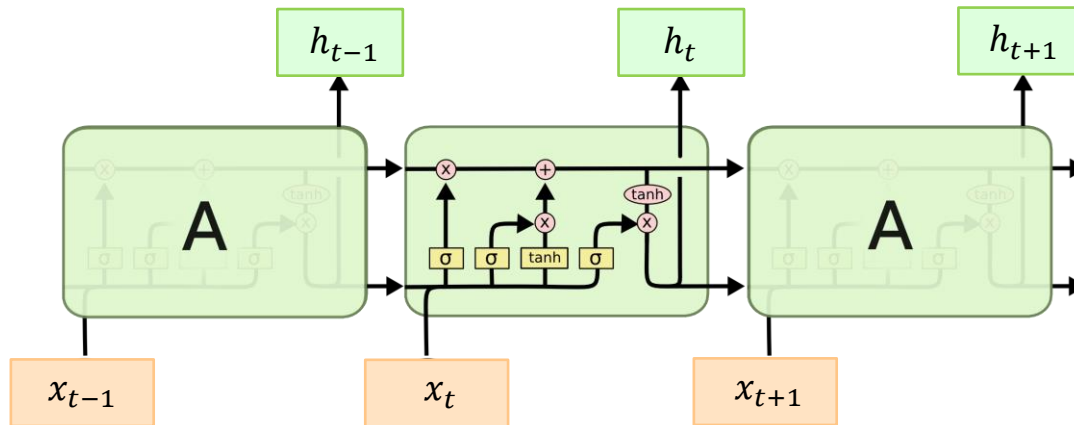
LSTM Recurrent Neural Network

□ LSTM architecture



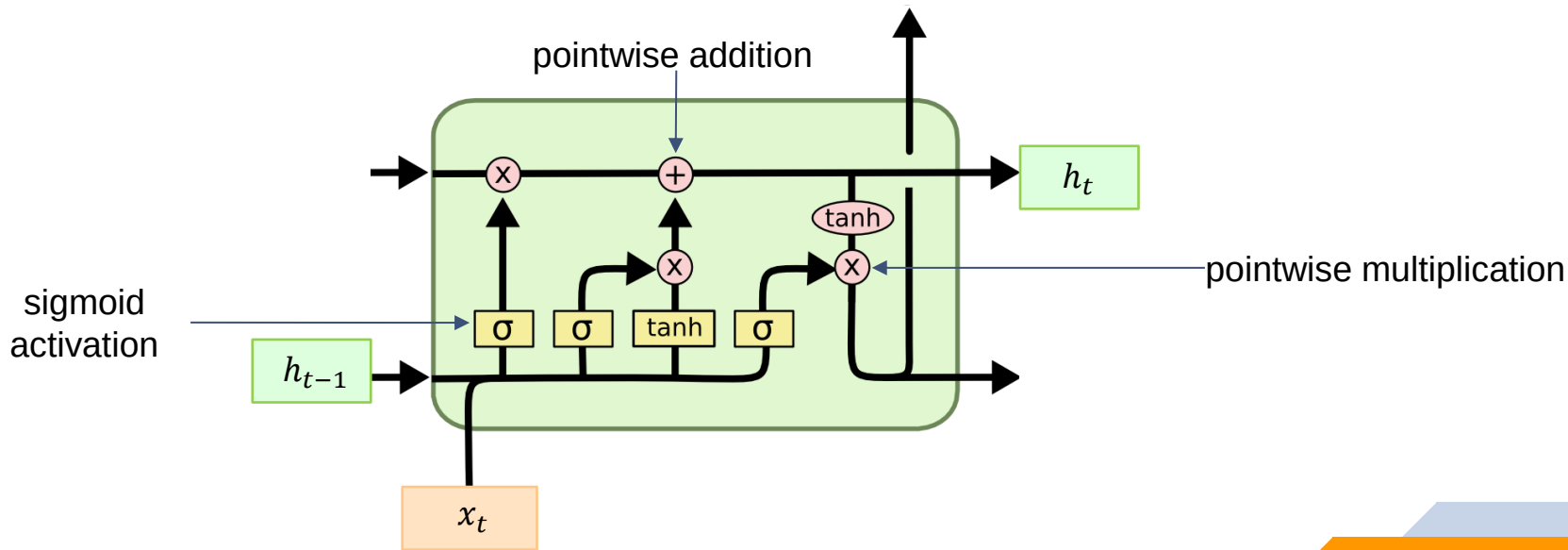
LSTM Recurrent Neural Network

□ LSTM architecture



LSTM Recurrent Neural Network

□ LSTM architecture



LSTM Recurrent Neural Network

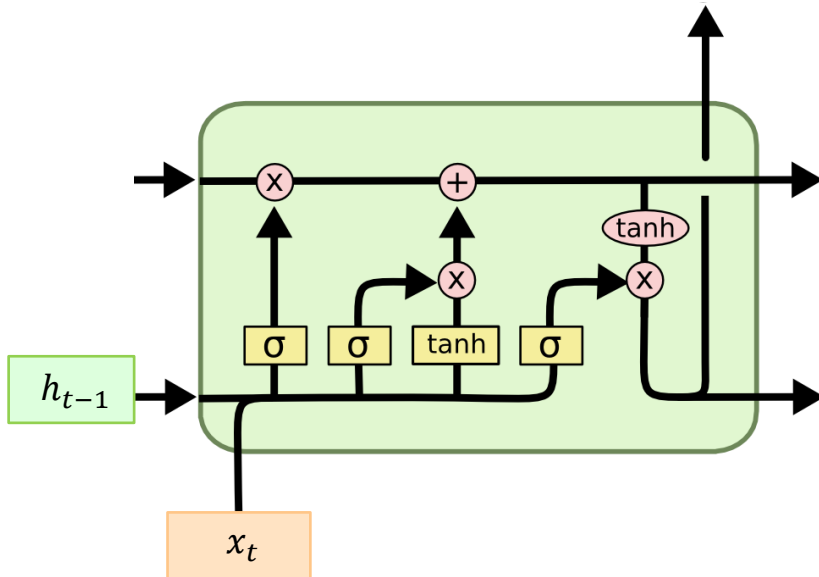
- ❑ The solution to the vanishing gradient problem is to use long-short-term-memory.
- ❑ LSTMs are more complex hidden layer units (i.e. not simple nonlinear functions) that can explicitly keep track of different kinds of information over arbitrarily long periods of time.
- ❑ Like simple RNNs, LSTMs use input representations and hidden layers.
- ❑ However, as well as hidden and input representations, they contain an explicit **memory component**.

LSTM Recurrent Neural Network

- ❑ **Key idea:** every time we encounter a word, we need to update the memory in an appropriate way.
- ❑ LSTM Cell operations:
 - Based on the **current input** and the **current hidden state**, what can we forget (remove from memory)
 - What new information should be stored in memory given **the current word** and **the hidden state**?
 - What should be included in the **new hidden state**?

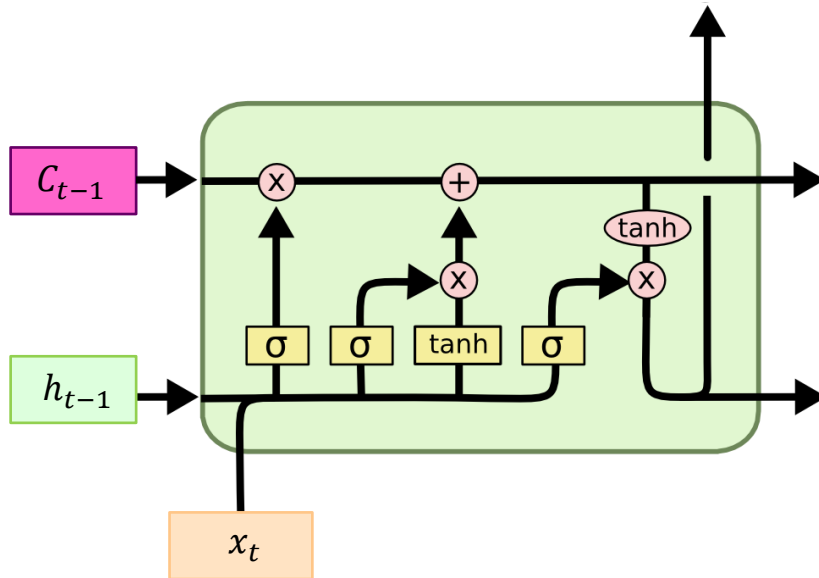
LSTM Cell

- Like simple RNNs, LSTMs use input representation and hidden layers:



LSTM Cell

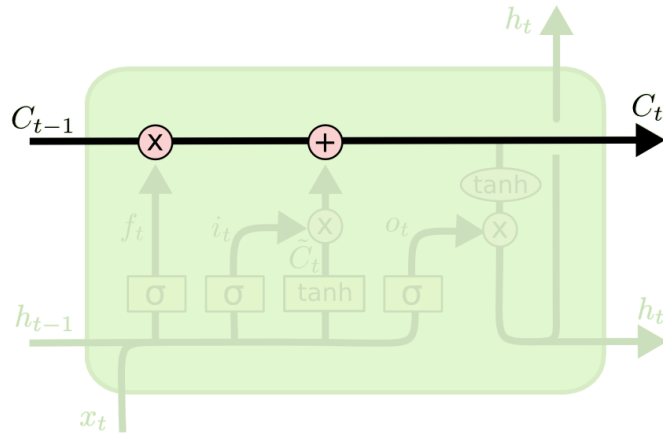
- However, as well as hidden and input representations, they contain an explicit memory component:



- The memory represents properties of what has gone before.
- There are different slots in this memory, which can represent information about what is in the preceding context.

LSTM Cell

- Key idea: every time we encounter a word, we need to update the memory in an appropriate way.



- The first step of this process is to decide what we can forget from memory, given the current word and the current hidden layer state.
- For example, if we encounter a verb, we may want to forget information about some other verb that occurred earlier in the sentence.

LSTM Cell

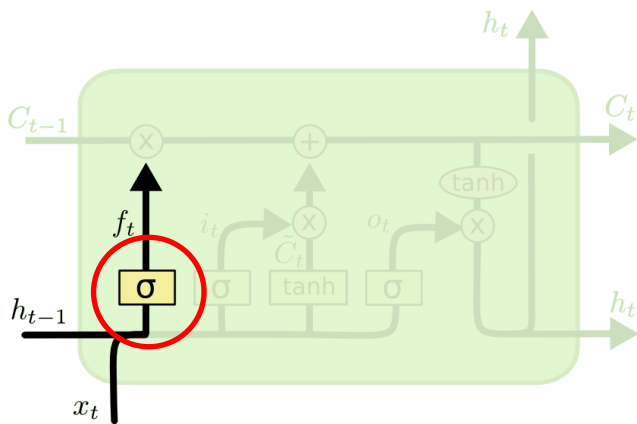
- ❑ LSTM uses gates to control information flow in the recurrent computations.
- ❑ Preserving the long term dependencies in the network is done by its Gating mechanisms.
- ❑ Gate is a Sigmoid neural network layer followed by point wise multiplication operator.
- ❑ Gates control the flow of information to/from the memory.
- ❑ Gates are controlled by a concatenation of the output from the previous time step and the current input

LSTM Cell

- ❑ There are three types of gates in LSTM cell:
 - ❑ Forget Gate
 - ❑ Input Gate
 - ❑ Output Gate

LSTM Cell

LSTM: Forget Gate

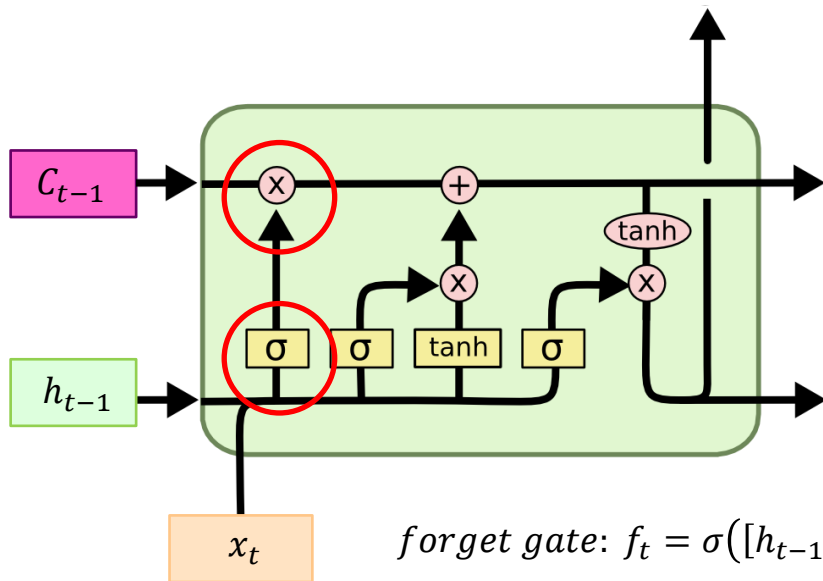


- The first step in LSTM is to decide what information we're going to throw away from the cell state
- This decision is made by a sigmoid layer called the **forget gate layer**
- You can imagine the forget gate as a kind of “filter” that removes certain parts of the memory.

$$\text{forget gate: } f_t = \sigma([h_{t-1}, x_t]W_f + b_f)$$

LSTM Cell

LSTM: Forget Gate

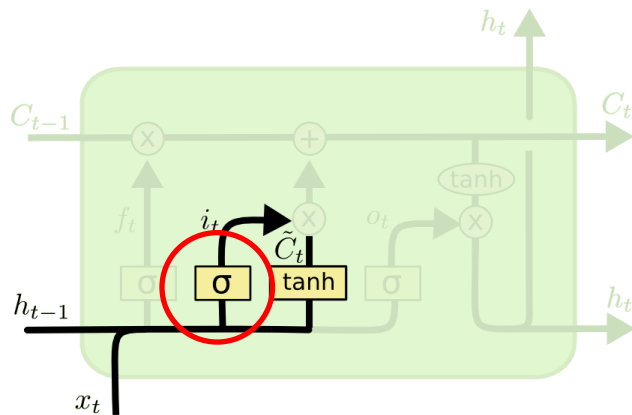


$$\text{forget gate: } f_t = \sigma([h_{t-1}, x_t]W_f + b_f)$$

- It looks at h_{t-1} and x_t , and outputs a number between 0 and 1 for each number in the cell state C_{t-1} .
- 1 represents “completely keep this” while a 0 represents “completely get rid of this.”
- The output of the forget gate is then multiply (elementwise) by the cell state

LSTM Cell

LSTM: Input Gate



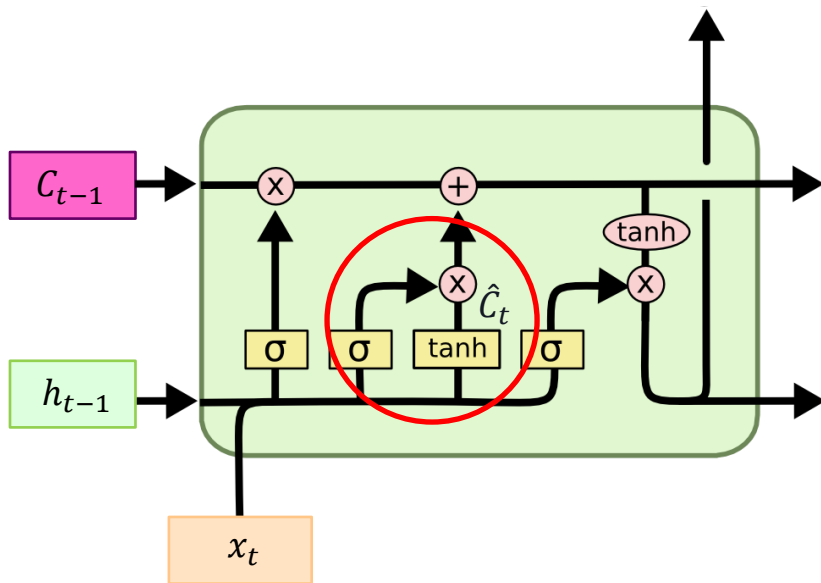
- The next step is to decide what new information should be put into memory, given the current word and hidden state.
- This decision is made by a sigmoid layer called the **input gate layer**
- As in case of the forget gate, the values to be changed in memory are determined as a function of the input and hidden state.

$$\text{input gate: } i_t = \sigma([h_{t-1}, x_t]W_i + b_i)$$

$$\hat{C}_t = \tanh([h_{t-1}, x_t]W_C + b_C)$$

LSTM Cell

LSTM: Input Gate

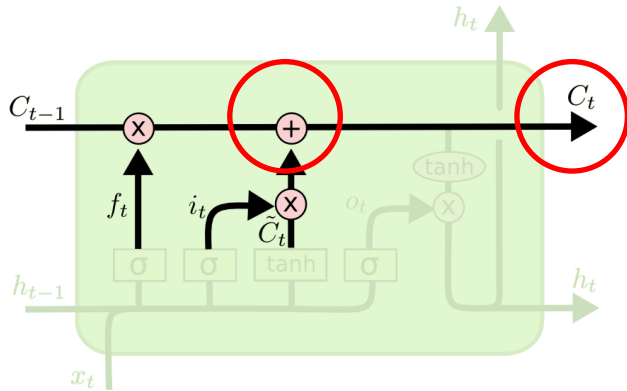


- First, input gate layer decides which values we'll update.
- Next, a $\tanh$ layer creates a vector of new candidate values, $\hat{C}_t$, that could be added to the state.
- Finally, these two are combined to create an update to the state.

$$\text{input gate: } i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \hat{C}_t = \tanh([h_{t-1}, x_t]W_C + b_C)$$

LSTM Cell

□ LSTM: Updating the old cell state (memory)

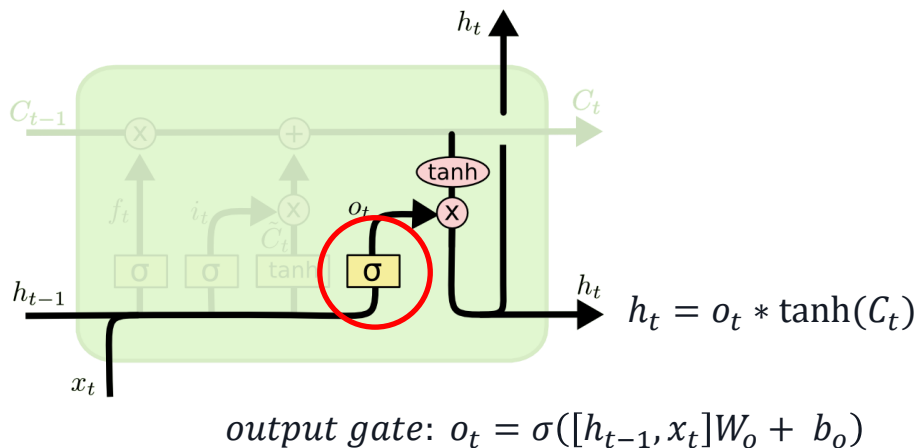


$$C_t = f_t * C_{t-1} + i_t * \hat{C}_t$$

- Next, element-wise summation is applied to merge the old memory and the current input.
- This gives us a new memory representation.
- The memory at word $t - 1$ has been updated to the memory at word t .

LSTM Cell

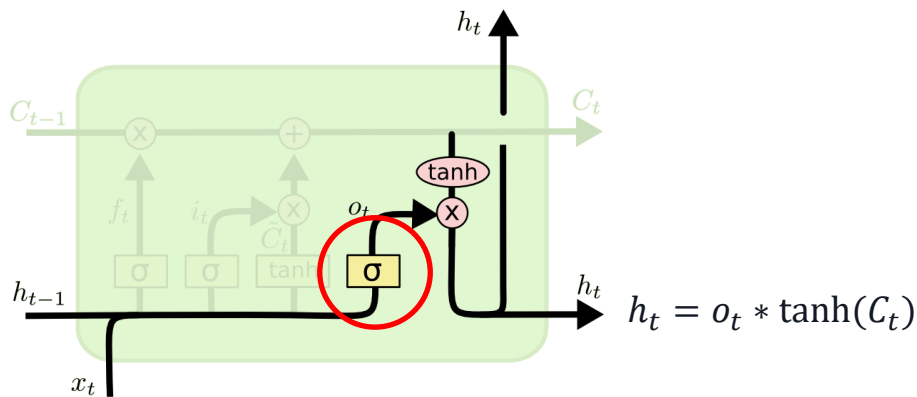
LSTM: Output Gate



- Finally we need to compute the **new hidden state** (output of this LSTM unit)
- This decision is made by a sigmoid layer called the **output gate layer**
- This is a function of the previous hidden state and the current input, along with the new memory representation.

LSTM Cell

LSTM: Output Gate

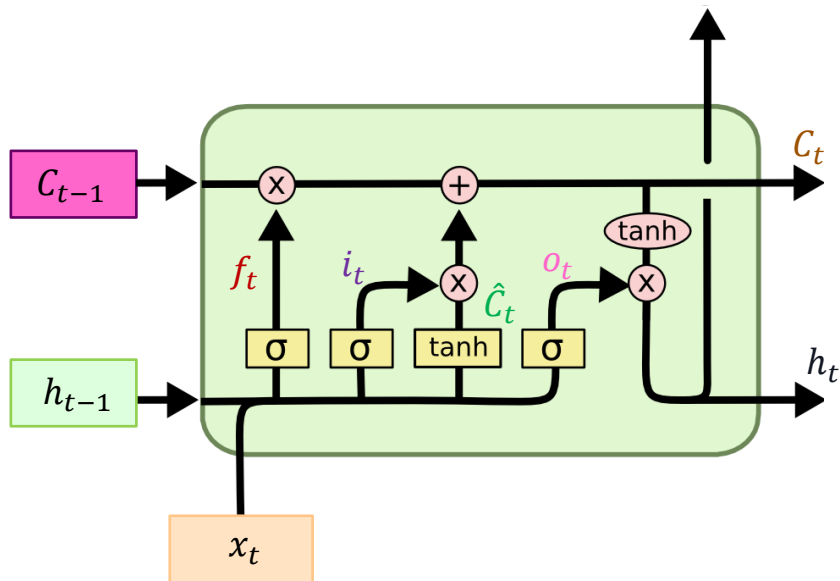


$$\text{output gate: } o_t = \sigma([h_{t-1}, x_t]W_o + b_o)$$

- First, we run a sigmoid layer which decides what parts of the cell state we're going to output.
- Then, we put the cell state through $\tanh$ and multiply it by the output of the sigmoid gate, so that we only output the parts we decided to.

LSTM Cell

- LSTM: Putting it all together



$$f_t = \sigma([h_{t-1}, x_t]W_f + b_f)$$

$$i_t = \sigma([h_{t-1}, x_t]W_i + b_i)$$

$$\hat{C}_t = \tanh([h_{t-1}, x_t]W_C + b_C)$$

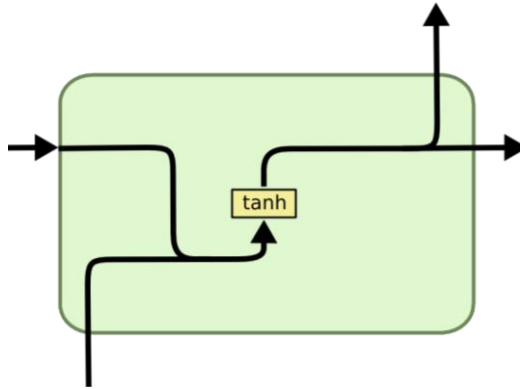
$$C_t = f_t * C_{t-1} + i_t * \hat{C}_t$$

$$o_t = \sigma([h_{t-1}, x_t]W_o + b_o)$$

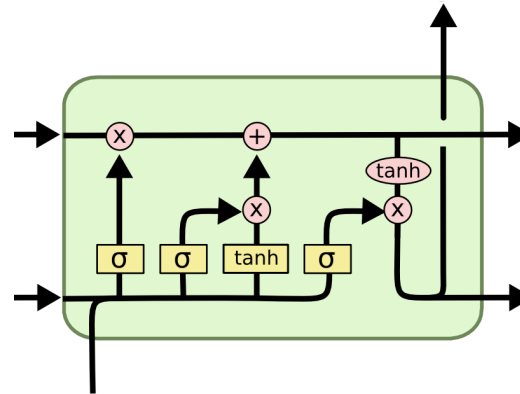
$$h_t = o_t * \tanh(C_t)$$

LSTM vs simple RNN

Simple RNN layer



LSTM layer



Why the complexity?

LSTM: Advantages

- ❑ What the LSTM network gives us is a way of **explicitly keeping relevant information available over time.**
- ❑ A particular part of the memory won't be changed unless it is explicitly changed by either the **forget gate** or the **input gate**.

LSTM: Advantages

- Backpropagation with LSTM

$$\frac{\partial L_t}{\partial W} = \frac{\partial L_t}{\partial y_t} \times \frac{\partial y_t}{\partial h_t} \times \frac{\partial h_t}{\partial c_t} \sum_{k=0}^t \left(\prod_{i=k+1}^t \frac{\partial c_i}{\partial c_{i-1}} \right) \frac{\partial c_k}{\partial W}$$

- Compactly, we can write c_t as:

$$c_t = c_{t-1} * f_t + \hat{c}_t * i_t$$

LSTM: Advantages

- Compactly, we can write c_t as:

$$c_t = c_{t-1} * f_t + \hat{c}_t * i_t$$

- $\frac{\partial c_t}{\partial c_{t-1}}$ will have much nicer properties than $\frac{\partial h_t}{\partial h_{t-1}}$ in Vanilla RNN
- It will allow to maintain a constant error, which will allow to continue learning over numerous time-steps and backpropagate through time and layers.

LSTM with Text: Keras Implementation

- ❑ Architecture:
 - ❑ Embedding layer (we still need inputs of the same length). To avoid the RNN model to be trained on the padded values, you can configure *mask_zero = True* in the Embedding layer.
 - ❑ LSTM layer (provides output for each time step or only for the last time step)
 - ❑ TimeDistributed layer (wrapper applies a layer to every temporal slice of an input)
 - ❑ Dense layer

Coming Next

Large Language Models